AI201 | Applications of AI Engineering

Applications of AI Engineering Summer 2026 (@ Section 2B | Thursday 5PM - 7PM PT)
Personal Member ID#: **131013**

Need help? Post on our class slack channel or email us at support@codepath.org

📬 Reminder: Project 3 is due by **Sunday, June 21st at 11:59PM PDT**.

Submit

Show What You Know: TakeMeter

⏰ **~9–11 hours total**

Online communities run on opinions. NBA subreddits, music theory Discord servers, reality TV recap forums, anime discussion boards — these are spaces where people share takes constantly, and the quality of those takes varies enormously. Some are insightful. Some are hyperbole. Some are just noise. But what makes a good take is genuinely hard to define — it's specific to the community, the context, and the moment.

In this project, you'll build **TakeMeter**: a fine-tuned text classifier that evaluates discourse quality in an online community of your choosing. You'll define the labels, collect and annotate the data, fine-tune a model, and then honestly assess where it works and where it falls apart.

The hardest part of this project isn't the training pipeline. It's the label

Week 1
▶ Module 2
Week 2
▶ Module 3
Week 3
Week 4

design — deciding what you're measuring, what counts as an example, and what to do with the cases that don't fit cleanly. By the end, you'll know more about fine-tuning from having wrestled with one messy dataset than from any amount of reading about it.

🎯 Goals

By completing this project, you will be able to:

- Design a label taxonomy for a classification task, including edge case handling.
- Curate and annotate a training dataset from real-world text.
- Run a fine-tuning pipeline and produce a usable model.
- Evaluate model performance with appropriate metrics and interpret failure modes.
- Articulate the gap between what a model learns and what you intended it to learn.

✅ Features

Required Features

- [ ] **Label taxonomy (2–4 labels):** Define 2–4 labels that capture meaningful distinctions in your chosen community's discourse. Labels must be:
  - Mutually exclusive: a post should belong to exactly one label
  - Exhaustive enough: you should be able to label at least 90% of posts without creating a catch-all "other" bucket
  - Grounded in community norms: the distinction should matter to people who actually participate in that community

  Document your labels, definitions, and examples in your `planning.md` and README.

- [ ] **Annotated dataset (at least 200 examples):** Collect and label at least 200 examples (posts or comments) from your chosen community. Split into train, validation, and test sets. In your README, document: where you collected the data, your labeling process, your label distribution (count per label), and at least 3 examples you found genuinely difficult to label and what you decided.
- [ ] **Fine-tuning pipeline:** Fine-tune `distilbert-base-uncased` (or another pre-trained model of your choice) on your labeled dataset. Your README must describe the model you started from, the training approach, and at least one key hyperparameter decision you made (e.g., learning rate, number of epochs, batch size).
- [ ] **Baseline comparison:** Compare your fine-tuned model to a zero-shot baseline: prompt Groq's `llama-3.3-70b-versatile` to classify each test example with no task-specific training. Your evaluation report must show both models' performance on the same test set.
- [ ] **Evaluation report:** Report performance metrics on your test set for both models. Include at minimum: overall accuracy for both models, at least one per-class metric (precision, recall, or F1), a confusion matrix or equivalent table, at least 3 specific examples the model got wrong with your analysis of why, and a reflection on what the model learned vs. what you intended it to learn.

Stretch Features

Complete any of these for extra credit. Update your `planning.md` before starting each one.

- [ ] **Inter-annotator reliability:** Have at least one other person label 30+ of your examples independently, and report your agreement rate (Cohen's kappa or simple percentage agreement). Analyze where you disagreed.
- [ ] **Confidence calibration:** Report whether your model's confidence scores are meaningful — does a 90% confident prediction actually get it right more often than a 60% confident one?
- [ ] **Error pattern analysis:** Go beyond listing individual wrong predictions — identify a systematic pattern in the errors (e.g., "the model consistently misclassifies sarcastic posts" or "it can't distinguish X from Y when the post is short").

- [ ] **Deployed interface:** Build a simple interface that accepts a new post, runs it through the classifier, and displays the label and confidence. Commit the interface code to your repo and document how to run it in your README.

💡 **Hints**

- Your labels are the most important design decision in this project. If your labels are vague, your model will learn something vague. Spend more time on label design than you think you need to.
- Label distribution matters. If 80% of your examples carry one label, your model will learn to predict that label constantly. Aim for at least 20% per label.
- The baseline comparison is not a formality — it tells you whether fine-tuning actually helped and by how much. Run it honestly.
- Wrong predictions are your most valuable data. The model's mistakes reveal what it actually captured, which is often not exactly what you intended.
- If your model is performing suspiciously well (>95% accuracy on a hard subjective task), check whether your test set leaked into training, or whether your labels are too easy.
- Collect your data before you start labeling. Read 30–40 examples first to check that your labels actually apply cleanly to real posts — you may need to revise them before you commit to annotating 200.

🛠️ Tools and Setup

This project uses a free tool stack — no paid subscriptions or API credits required beyond your existing Groq account.

Recommended stack

Component	Tool	Notes
Base model	`distilbert-base-uncased`	HuggingFace — free to download, no account needed

Fine-tuning	Google Colab (free GPU)	Free T4 GPU; fine-tuning DistilBERT on 200 examples takes ~5–15 min
Training libraries	`transformers` + `datasets` + `scikit-learn`	Pre-installed on Colab — no setup needed
Baseline LLM	Groq (`llama-3.3-70b-versatile`)	Free tier — same account as Projects 1–2

Getting started

- [ ] **Make a copy of the starter Colab notebook.** Go to the TakeMeter starter notebook and click **File** → **Save a copy in Drive**. This gives you your own editable copy. The notebook handles the training loop, metrics, and confusion matrix — your job is to fill in your label map, upload your CSV, and write your Groq classification prompt.
- [ ] **Set your runtime to T4 GPU.** In your copy, go to **Runtime** → **Change runtime type**, select **T4 GPU**, and click Save. Do this before running any cells.
- [ ] **Add your Groq API key using Colab Secrets** (recommended over pasting it directly):
  - Click the 🔑 icon in the left sidebar
  - Add a secret named `GROQ_API_KEY` with your key as the value
  - Enable notebook access for the secret
  - The notebook's baseline section has instructions for using it
- [ ] **Create a GitHub repository** for everything that lives outside the notebook: your `planning.md`, labeled dataset CSV, README, and the output files you'll download from Colab (`evaluation_results.json`, `confusion_matrix.png`). Name it something like `ai201-project3-takemeter`.

Milestone 1: Choose Your Community and Define Your Labels

⏰ **~45 min**

Before touching any code or data, make two decisions: what community you'll study, and what you'll measure. These decisions constrain everything downstream — your data sources, your annotation effort, your model's learning task, and what your evaluation actually tells you. Getting them right at the start saves hours of rework later.

▼ 🧠 Before You Start

📒 Strong vs. Weak Label Taxonomies

The single most important design decision in this project is your labels. Weak labels produce models that learn nothing useful. Here's what the difference looks like:

❌ **Weak taxonomy (too vague):**

> - `good` — a quality post
> - `bad` — a low-quality post

This fails because "quality" is entirely in the eye of the beholder. Two annotators will apply it differently, and the model will learn an inconsistent signal. The boundary is also impossible to specify: what makes one reaction post "good" and another "bad"?

✅ **Strong taxonomy (precise and grounded):**

> - `analysis` — the post makes a structured argument backed by statistics, historical comparison, or tactical observation. Evidence is specific and verifiable.
> - `hot_take` — a bold, confident opinion stated without supporting evidence. The claim might be true, but the post asserts rather than argues.
> - `reaction` — an immediate emotional response to a specific event. Little to no argument — the post is expressing a feeling in the moment.

These work because: (1) you can state the decision boundary in a sentence, (2) two people reading the definitions would agree on most examples, and (3) the distinctions reflect how people in the community actually talk about discourse quality.

What makes labels too vague:

- Relying on subjective words without definition: "insightful," "toxic," "engaging"
- Labels that overlap: if a post can reasonably belong to two labels, the boundary isn't precise enough
- Labels too broad to apply: if one label captures 80% of posts, it's not distinguishing anything

📙 Handling Ambiguous Posts

Some posts genuinely sit at the boundary between two labels. That's not a flaw — it's a signal that your taxonomy needs an explicit decision rule for that case.

Example ambiguous post (for an NBA community):

> "LeBron is overrated — his playoff win rate against top-seeded opponents is below .500."

Is this `hot_take` (bold claim, accusatory framing) or `analysis` (cites a specific stat)?

Decision rule: If the post provides specific, verifiable evidence that would support the claim even if you removed the opinion framing, label it `analysis`. If the evidence is vague, cherry-picked, or decorative — just enough to sound credible but not genuinely reasoning — label it `hot_take`. The one-stat post above is borderline; the framing is accusatory and the stat is selected for effect rather than as part of an argument. → `hot_take`.

Before you commit to your labels, find one post like this in your community. Write down which labels it could belong to, and write the decision rule you'll use. This is the most valuable design work you'll do before annotating 200 examples.

Close Section

- [ ] Choose one online community where discourse is active, text-heavy, and varied in quality. Strong options include: sports subreddits (r/nba, r/soccer, r/fantasyfootball) where "hot take vs. analysis" is a real distinction; music communities (r/LetsTalkMusic, r/indieheads) where "opinion vs. argument" matters; TV and film fan spaces (r/television, r/TrueFilm) where discourse ranges from reaction to critique; gaming communities (r/smashbros, r/leagueoflegends) where you can distinguish tips/strategy from hype. Whatever you choose, you need to be able to collect at least 200 public posts or comments from it.
- [ ] Read 30–40 posts from your chosen community before committing to labels. Don't design labels from memory — read the actual text and see what patterns emerge. Ask: what makes some posts more substantive than others here? What would a regular in this community recognize as a good vs. bad take?
- [ ] Define 2–4 labels. For each label, write: a one-sentence definition, 2 example posts that clearly belong to it, and 1 post you're not sure about. The uncertain case is the most important — it forces you to sharpen the boundary between labels before you've annotated 200 examples.
- [ ] Check your labels for mutual exclusivity: can you pick a random post and assign it to exactly one label without ambiguity most of the time? If two labels overlap significantly, merge them or redefine the boundary.
- [ ] Write a 2–3 sentence description of your community, your labels, and why these distinctions matter to people in that community. You'll use this in your `planning.md`.

📍 Checkpoint

You can state your 2–4 labels with one-sentence definitions and 2 concrete examples each. You can name the hardest anticipated edge case — a type of post that genuinely sits between two labels — and explain how you'll handle it. If you can't name a hard case, you haven't thought hard enough about your label boundaries yet.

Milestone 2: Write Your Spec Before Any Code

⏰ **~45 min**

Write your `planning.md` before you collect a single labeled example. There is no template for this document — you design the structure yourself. That's intentional: by this point in the course, you've seen two planning documents. Now you're responsible for deciding what to plan and how to organize it. What matters is that you address the six questions below with real, specific answers — not placeholders.

- [ ] Create `planning.md` in your repo root. Design the section structure yourself. At minimum, your document must substantively address these six questions:
    - **Community:** What community did you choose and why? Why is this community a good fit for a classification task — what makes the discourse varied enough to be interesting?
    - **Labels:** What are your 2–4 labels? Define each in a complete sentence. Include 2 example posts per label.
    - **Hard edge cases:** What type of post will be genuinely ambiguous between two labels? How will you handle it when you encounter it during annotation?
    - **Data collection plan:** Where will you collect examples? How many per label? What will you do if a label is underrepresented after 200 examples?
    - **Evaluation metrics:** Which metrics will you use to evaluate your model and why are those the right ones for this specific task? (Accuracy alone is not enough — explain what else you need and why.)
    - **Definition of success:** What performance would make this classifier genuinely useful? What would you accept as "good enough" for deployment in a real community tool?
- [ ] Review your evaluation plan: are your success criteria specific enough that you could objectively determine at the end whether you hit them?

- [ ] Add an **AI Tool Plan** section to your `planning.md`. This project's workflow is different from implementation projects — there's no code to generate, so your plan should cover the three places where AI tools actually help here:
    - **Label stress-testing:** Give the AI your label definitions and edge case description, and ask it to generate 5–10 posts that sit at the boundary between two labels. If it produces posts you can't classify cleanly, your definitions need tightening — do that now, before you annotate 200 examples.
    - **Annotation assistance:** Decide whether you'll use an LLM to pre-label a batch of examples before reviewing them yourself. If yes, note which tool you'll use and how you'll track which examples were pre-labeled (for disclosure in your AI usage section).
    - **Failure analysis:** Plan to give your list of wrong predictions to an AI tool and ask it to identify patterns before you write up your evaluation. Note what you'll look for and how you'll verify the patterns yourself.
- [ ] Update `planning.md` before starting any stretch features later.

📍 **Checkpoint**

`planning.md` contains all six required questions with substantive, specific answers. Your label definitions are precise enough that two people reading them would agree on most examples. Your success criteria define a specific performance threshold, not just "it should work well." Your AI Tool Plan section covers label stress-testing, annotation assistance, and failure analysis — even if you plan to skip some of these, you've made an explicit decision about each.

Milestone 3: Collect and Annotate Your Dataset

⏰ **~3–4 hours**

Data collection and annotation is the most time-consuming part of this project — and the most important. The model can only learn what your labels actually capture, which means bad annotation leads to a bad model regardless of your training setup. Work carefully: read each example, apply your label definition, and document every case that gave you pause.

- [ ] Collect at least 200 posts or comments from your chosen community. Use public posts only — no private channels or content behind authentication. Reddit, public Discord servers, fan wikis, and sports comment sections are all valid sources. You can collect manually (copy-paste into a spreadsheet) or use a scraping tool if you're comfortable with it, but don't let data collection become a coding project — manual collection of 200 examples takes about 1–2 hours and keeps you close to the data.
- [ ] Save your collected examples in a **CSV file** with at minimum two columns: `text` (the post or comment) and `label` (your string label). Include a third column for any notes about difficult cases. The notebook expects a CSV — save one complete labeled file, not multiple pre-split files. The notebook handles the train/validation/test split automatically (70% / 15% / 15%). This file goes in your repo.
- [ ] Optionally, use an LLM to pre-label a batch of examples before reviewing them yourself — provide it your label definitions from `planning.md` and a set of unlabeled posts and ask it to assign one label per post. Pre-labeling can speed up annotation, but you must review and correct every pre-assigned label. Skimming without genuine review defeats the purpose and produces noisy training data. If you use this workflow, disclose it in your AI usage section.
- [ ] Label each example using your definitions from `planning.md`. Do not label in bulk by skimming — read each post. As you go, keep a running list of cases that gave you genuine pause: what the post was, which labels it could belong to, and what you decided.
- [ ] After labeling, count your examples per label. If any label accounts for more than 70% of your dataset, you have an imbalance problem — collect more examples from the underrepresented labels before moving on. A heavily imbalanced dataset will produce a model that predicts the majority class most of the time.

📍 **Checkpoint**

You have at least 200 labeled examples saved in a single CSV file (not pre-split). Your label distribution shows no single label above 70% of the dataset. You have documented at least 3 specific examples that were difficult to label and what you decided — these go in your `planning.md` (in the Hard Edge Cases section you already started in Milestone 2).

Milestone 4: Run Your Baseline

⏰ **~1 hour**

Before fine-tuning, establish a baseline so you know what you're trying to beat. A zero-shot LLM prompt is a legitimate and informative baseline — it tells you how hard the task is for a general model with no training, which gives your fine-tuned model's performance real meaning. Run this on your locked test set now, before you touch your training data.

⚙️ **Notebook setup — run this before the baseline:** Section 5 (the baseline) depends on variables created in earlier sections. You need to run Sections 1 and 2 first, even though you're not fine-tuning yet.

- [ ] Open your copy of the starter Colab notebook. Confirm the runtime is set to **T4 GPU** before running any cells.
- [ ] Run **Section 1**: define your label map and upload your labeled CSV. The notebook will prompt you to upload a file from your computer — no manual placement in Colab or Google Drive is required. Upload your single labeled CSV; the notebook handles the train/validation/test split automatically. If your Colab session disconnects or resets, you'll need to re-upload the file and re-run Sections 1 and 2.
- [ ] Run **Section 2**: the notebook splits your dataset (70% / 15% / 15%) and tokenizes all splits. Review the split sizes and label distribution to confirm they look reasonable.

- [ ] Open **Section 5** of the notebook. Add your Groq API key (via Colab Secrets or directly in the cell — never commit it to GitHub) and write your classification prompt in the provided cell. Your prompt should include your label definitions as written in `planning.md` and instruct the model to output only the label name — the notebook's parsing logic depends on a clean, consistent response.
- [ ] Run the baseline cells. The notebook will classify every example in your test set, print overall accuracy and per-class metrics, and flag any responses it couldn't parse. If more than ~10% of responses are unparseable, revise your prompt to make the expected output format clearer.
- [ ] Reflect briefly: where did the baseline struggle? Are there specific labels it consistently confuses? Write down your hypothesis — you'll test it after fine-tuning.

📍 **Checkpoint**

You have baseline performance numbers on your test set: overall accuracy and at least a per-class breakdown. You have not yet looked at the fine-tuned model's performance on this test set (you haven't trained it yet). The baseline results are saved somewhere you can reference them in the evaluation report.

Milestone 5: Fine-Tune Your Model

⏰ **~1.5–2 hours**

Fine-tune `distilbert-base-uncased` on your training data using Google Colab's free T4 GPU. The training itself is fast — expect 5–15 minutes for 200 examples — but setting up the pipeline correctly takes time. Follow the steps below carefully. Once training is done, evaluate on your test set and compare to the baseline.

▼ 🧠 Before You Start

📙 Reading Evaluation Output

The notebook produces several evaluation outputs. Here's how to interpret them.

Overall accuracy: fraction of test examples the model got right. A baseline of 33% on a 3-class task is trivial — a random guesser matches that. A fine-tuned model should meaningfully exceed the baseline.

Per-class metrics (precision, recall, F1):

- **Precision** for a label = "of all examples the model *predicted* as this label, what fraction actually were?" High precision, low recall means the model is conservative — it only predicts this label when confident, but misses many real examples.
- **Recall** for a label = "of all examples that *truly are* this label, what fraction did the model catch?" High recall, low precision means the model over-predicts this label.
- **F1** = harmonic mean of precision and recall. This is the most useful single number per class.

What good vs. poor performance looks like:

Scenario	What it means
All per-class F1 ≥ 0.70	Model is learning all distinctions well
One class F1 ≈ 0, others fine	Model can't learn that boundary — check labels and examples
All classes F1 similar and low	Task may be too hard for 200 examples, or labels are inconsistent
Fine-tuned barely beats baseline	Fine-tuning added little — labels may be too easy or too noisy

Confusion matrix: rows are true labels, columns are predicted labels. The diagonal is correct predictions. Off-diagonal cells show which labels the model confuses and in which direction. A cell with a large number at position (row=`analysis`, col=`hot_take`) means the model is predicting `hot_take` when the true label is `analysis` — a specific, actionable pattern.

Close Section

⚙️ **Sections 1, 2, and 5 are already done** from Milestone 4 (you ran the baseline in Section 5 there). Do not re-run them unless your Colab runtime has reset. If it has: re-upload your CSV, re-run Sections 1 and 2, and re-run Section 5 (re-add your Groq key + prompt) — Section 6's comparison and export need your baseline numbers from Section 5. Resetting the runtime without re-running these will cause errors.

- [ ] Run **Section 3**: the notebook loads `distilbert-base-uncased` and fine-tunes it on your training data. Training takes **5–15 minutes** on a T4 GPU. The default settings are 3 epochs, learning rate 2e-5, batch size 16. If you adjust any hyperparameters, note what you changed and why — this goes in your README.
- [ ] Run **Section 4**: the notebook evaluates the fine-tuned model on your test set, prints per-class metrics, and generates a confusion matrix image (`confusion_matrix.png`). Review the wrong predictions — pick 3 to analyze in depth for your README.
- [ ] Run **Section 6**: the notebook prints the side-by-side baseline-vs-fine-tuned comparison and writes `evaluation_results.json`. (This uses your baseline numbers from Section 5 — if your runtime reset since Milestone 4, re-run Sections 1, 2, and 5 first.)
- [ ] Download `evaluation_results.json` and `confusion_matrix.png` from Colab (Files panel → right-click → Download) and commit them to your GitHub repo. In Milestone 6 you'll also write the confusion matrix out as a markdown table in your README — that text version is the one that needs to read cleanly in your report, with the committed `confusion_matrix.png` kept as a supplementary copy.

📍 Checkpoint

Fine-tuning completed without error. You have evaluation results on your test set for the fine-tuned model. You can compare these directly to your baseline results from Milestone 4. If the fine-tuned model performs worse than the baseline across the board, that's a signal worth investigating before writing up your report — check for label leakage, class imbalance, or a training bug.

Milestone 6: Evaluate, Document, and Record

⏰ **~1–2 hours**

Write your evaluation report, complete your README, and record your demo. The hardest intellectual work in this milestone is the analysis: honestly explaining where your model fails and what that reveals about your labels, your data, or the task itself. A model that "mostly works" but can't be understood is less valuable than one that reveals specific, diagnosable failure modes.

📂 **Which file does what:**

- `planning.md` = your *design thinking before and during the project* — label definitions, edge case rules, data collection plan, evaluation metrics reasoning, AI tool plan, and hard annotation decisions. You wrote most of this in Milestones 1–2.
- `README.md` = your *final polished documentation for a reader* — a summary of what you built, your evaluation results, and your analysis. It can reference or lightly repeat content from `planning.md`, but it's written for someone who hasn't read that document. Think of `planning.md` as your working notes and `README.md` as your final report.

You do not need to duplicate everything. The README should be complete on its own; `planning.md` can have the detail and working notes behind each decision.

- [ ] Before writing your analysis, use an AI tool to help surface patterns in your wrong predictions. Paste your misclassified examples into Claude or another LLM and ask it to identify any common themes — similar post length, use of sarcasm, a specific label pair that keeps getting confused, short or low-information posts, etc. Then verify those patterns yourself by re-reading the examples. This step doesn't replace your analysis; it helps you see patterns that are easy to miss when reviewing cases one at a time. Include whatever you find — and whatever you had to correct or discard — in your evaluation report.

- [ ] Write your evaluation report directly in the README. Include: overall accuracy for both models, per-class metrics for both models, a confusion matrix for your fine-tuned model written out as a markdown table directly in the README (not only the `confusion_matrix.png` you committed — write the matrix out as text so it reads cleanly in the README), and at least 3 specific examples the fine-tuned model got wrong with your analysis of why each one failed. "The model got it wrong" is not analysis — use these guiding questions to go deeper:

    - **Which labels are being confused?** Look at the confusion matrix — is there one pair of labels (e.g., `analysis` → `hot_take`) that accounts for most of the errors? A directional pattern tells you exactly which boundary the model hasn't learned.

    - **Why is that boundary hard?** Is it ambiguous language, sarcasm, short posts, or posts where the topic signals one label but the structure signals another?

    - **Is this a labeling problem or a prompt/data problem?** If you labeled those examples consistently but the model still gets them wrong, the issue is in your training data distribution or the boundary itself. If you find you labeled similar posts differently, the issue is annotation inconsistency.

    - **What would need to change to fix it?** More examples for the confused class? A tighter label definition? More diverse examples that show the hard case explicitly?

- [ ] Include a **Sample Classifications** subsection in your evaluation report: a markdown table or list of 3–5 example posts run through your fine-tuned model, each shown with the predicted label and its confidence score, and for at least one correctly-predicted example, a sentence explaining why the prediction is reasonable. Write these out as text (a markdown table or list) — not a screenshot — so they read cleanly in the README.
- [ ] Write a reflection on what your model captured vs. what you intended it to capture. This is distinct from listing wrong predictions — it's a higher-level observation about the gap between your label definitions and what the model's decision boundary actually captures. What did the model overfit to? What did it miss?
- [ ] Write your README covering all required sections from the submission checklist. Every section should be substantive — a grader should be able to understand your design decisions from reading it, not just that you completed the steps.
- [ ] Write your spec reflection in the README: describe one way the spec helped guide your implementation and one way your implementation diverged from it and why.
- [ ] Add the AI usage section to your README. Describe at least 2 specific instances: what you directed the AI tool to do, what it produced, and what you changed or overrode. If you used any AI assistance during annotation (e.g., asking an LLM to pre-label examples you then reviewed), disclose it here.
- [ ] Record a 3–5 minute demo video. Show: 3–5 posts being classified by your fine-tuned model with label and confidence visible, one correct prediction with narration of why it's reasonable, one incorrect prediction with narration of what went wrong, and a brief walkthrough of your evaluation report.

📍 Checkpoint

Evaluation report documents both models with per-class metrics, a confusion matrix, and 3 analyzed failures. Reflection addresses the gap between intended and learned behavior specifically. README covers all required sections. Demo video is recorded and shows all required moments.

📬 Submitting Your Project

Submit all of the following through the Course Portal:

- [ ] Link to your GitHub repository
- [ ] `planning.md` in your repo root (written before data collection, updated before stretch features)
- [ ] Your labeled dataset (CSV or JSON) included in the repo or linked from the README
- [ ] `README.md` including:
  - Community choice and reasoning
  - Label taxonomy: definitions and 2 examples per label
  - Data collection source, labeling process, label distribution, and 3 difficult-to-label examples with your decisions
  - Fine-tuning approach: base model, training setup, and at least one hyperparameter decision
  - Baseline description: prompt used and how results were collected
  - Full evaluation report: metrics for both models (accuracy + per-class), confusion matrix (written as a markdown table in the README, not only the committed image), 3 specific wrong predictions with analysis, and a sample-classifications table (3–5 posts with predicted label and confidence, one correct example explained)
  - Reflection: what the model learned vs. what you intended
  - Spec reflection: one way the spec helped you, one way implementation diverged from it and why
  - AI usage section: at least 2 specific instances describing what you directed the AI to do and what you revised or overrode; disclose any annotation assistance

- [ ] Demo video (3–5 minutes) showing:
    - 3–5 posts being classified by your fine-tuned model with label and confidence visible
    - One correct prediction narrated with explanation
    - One incorrect prediction narrated with explanation of why it went wrong
    - Brief walkthrough of your evaluation report

🗺️ How It's Graded

A detailed breakdown of graded features and points can be found on the course grading page.